COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/16/2026	**DATE SUBMITTED:**	9/23/2026
NAME:	Katindig, Jiro Luis	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 Tree ADT

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 9-1: General Tree

Code Snippet

ACTIVITY 9 > C Generaltree.h > ...

```
#ifndef GENERALTREE_H
#define GENERALTREE_H

#include <iostream>

template <typename T>
class GeneralTree {
private:
    T key;
    GeneralTree<T>* firstChild;
    GeneralTree<T>* nextSibling;

public:
    //constructor
    GeneralTree(T rootObj);

    //inserting a child node, gets added after the existing ones and returns the new node
    GeneralTree<T>* insertChild(T newNode);

    GeneralTree<T>* getFirstChild();
    GeneralTree<T>* getNextSibling();

    T getRootVal();
    void setRootVal(T obj);

    bool isLeaf();

    //just to check the tree looks right
    void printTree(int depth = 0);
};

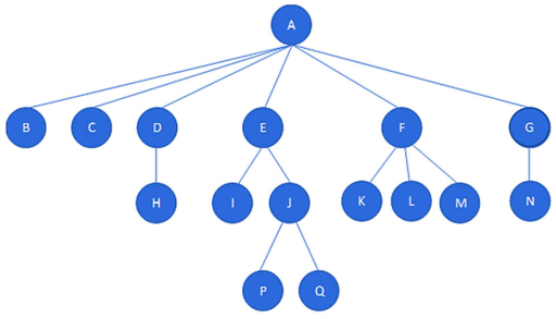
template <typename T>
GeneralTree<T>::GeneralTree(T rootObj) {
    key = rootObj;
    firstChild = nullptr;
    nextSibling = nullptr;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::insertChild(T newNode) {
    GeneralTree<T>* newChild = new GeneralTree<T>(newNode);

    if (firstChild == nullptr) {
        //no kids yet so this is the first one
        firstChild = newChild;
    } else {
```

```
46      } else {
47          //otherwise go to the end of the sibling list and stick it there
48          GeneralTree<T>* temp = firstChild;
49          while (temp->nextSibling != nullptr) {
50              temp = temp->nextSibling;
51          }
52          temp->nextSibling = newChild;
53      }
54
55      return newChild;
56  }
57
58  template <typename T>
59  GeneralTree<T>* GeneralTree<T>::getFirstChild(){
60      return firstChild;
61  }
62
63  template <typename T>
64  GeneralTree<T>* GeneralTree<T>::getNextSibling(){
65      return nextSibling;
66  }
67
68  template <typename T>
69  T GeneralTree<T>::getRootVal() {
70      return key;
71  }
72
73  template <typename T>
74  void GeneralTree<T>::setRootVal(T obj) {
75      key = obj;
76  }
77
78  template <typename T>
79  bool GeneralTree<T>::isLeaf() {
80      return firstChild == nullptr;
81  }
82
83  template <typename T>
84  void GeneralTree<T>::printTree(int depth) {
85      for (int i = 0; i < depth; i++) {
86          std::cout << "  ";
87      }
88      std::cout << key << std::endl;
89
90      if (firstChild != nullptr) {
91          firstChild->printTree(depth + 1); //go down
92      }
93      if (nextSibling != nullptr) {
94          nextSibling->printTree(depth); //next one over
95      }
96  }
97
98  #endif
```

Tree Diagram



Observation	I traversed the tree recursively, computing each node's height from its children and its depth from its parent.

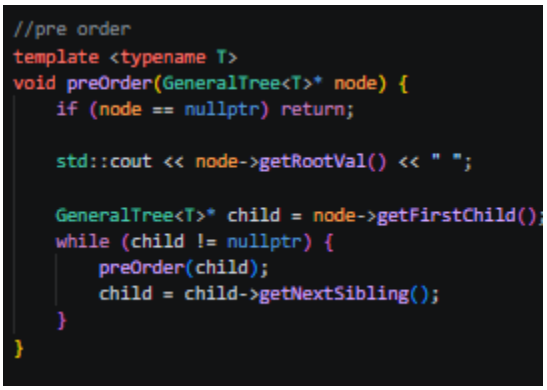
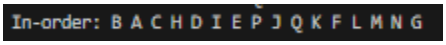
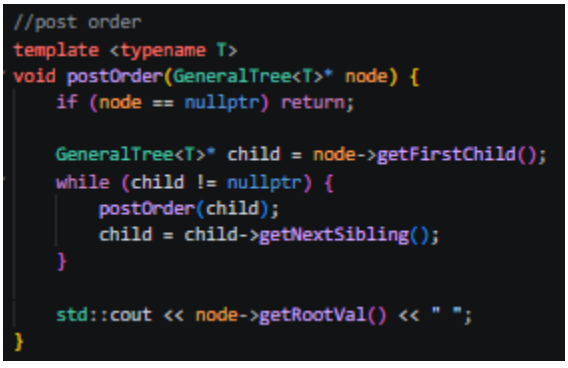
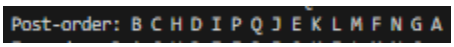
Table 9-2

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9-3

Pre-order	ABCDHEIJPQFKLMGN
In-order	BACHDIEPJQKFLMNG
Post-order	BCHDIPQJEKLMFNGA

Table 9-4

Pre-order Snippet	
Pre-order Output	
In-order Snippet	
In-order Output	
Post-order Snippet	
Post-order Output	

Observation	I wrote pre-order, in-order, and post-order recursively, for in-order I recurse into the first child, print the node, then recurse into the remaining siblings, since a general tree node can have more than two children.

Table 9-5

Code Snippet

```
static bool foundFlag = false;


template <typename T>
void searchHelper(GeneralTree<T>* node, std::string choice, T key) {
    if (node == nullptr || foundFlag) return;

    if (choice == "PRE") {
        if (node->getRootVal() == key) {
            std::cout << key << " was found" << std::endl;
            foundFlag = true;
            return;
        }
        GeneralTree<T>* child = node->getFirstChild();
        while (child != nullptr && !foundFlag) {
            searchHelper(child, choice, key);
            child = child->getNextSibling();
        }
    }
    else if (choice == "POST") {
        GeneralTree<T>* child = node->getFirstChild();
        while (child != nullptr && !foundFlag) {
            searchHelper(child, choice, key);
            child = child->getNextSibling();
        }
        if (!foundFlag && node->getRootVal() == key) {
            std::cout << key << " was found" << std::endl;
            foundFlag = true;
        }
    }
    else if (choice == "IN") {
        GeneralTree<T>* firstChild = node->getFirstChild();
        if (firstChild != nullptr && !foundFlag) {
            searchHelper(firstChild, choice, key);
        }
        if (!foundFlag && node->getRootVal() == key) {
            std::cout << key << " was found" << std::endl;
            foundFlag = true;
        }
        if (firstChild != nullptr && !foundFlag) {
            GeneralTree<T>* nextChild = firstChild->getNextSibling();
            while (nextChild != nullptr && !foundFlag) {
                searchHelper(nextChild, choice, key);
                nextChild = nextChild->getNextSibling();
            }
        }
    }
}
```

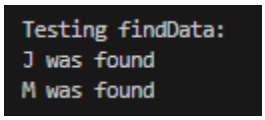
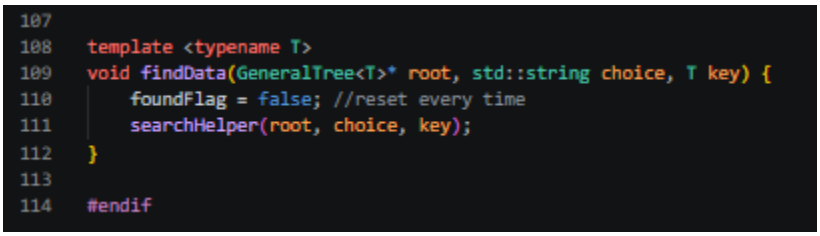
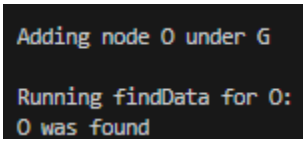
Output	
Observations	searchHelper walks the tree the same way the traversal functions do, but only prints when it hits a matching key. A static foundFlag gets set once the key is found, so the recursion stops early instead of checking the rest of the tree.

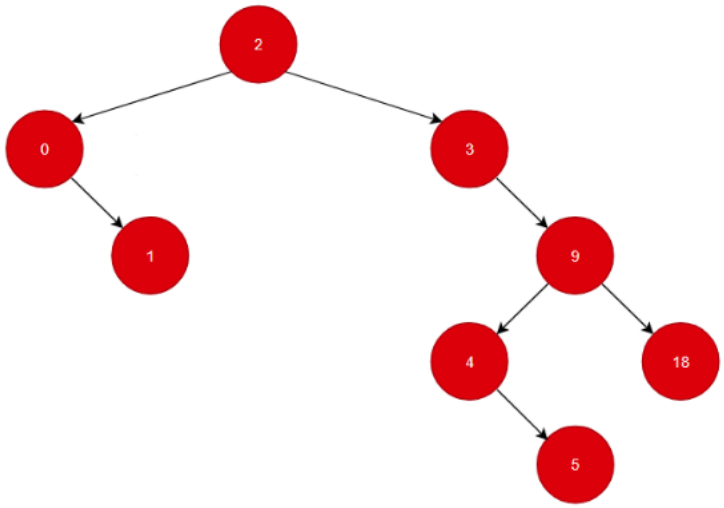
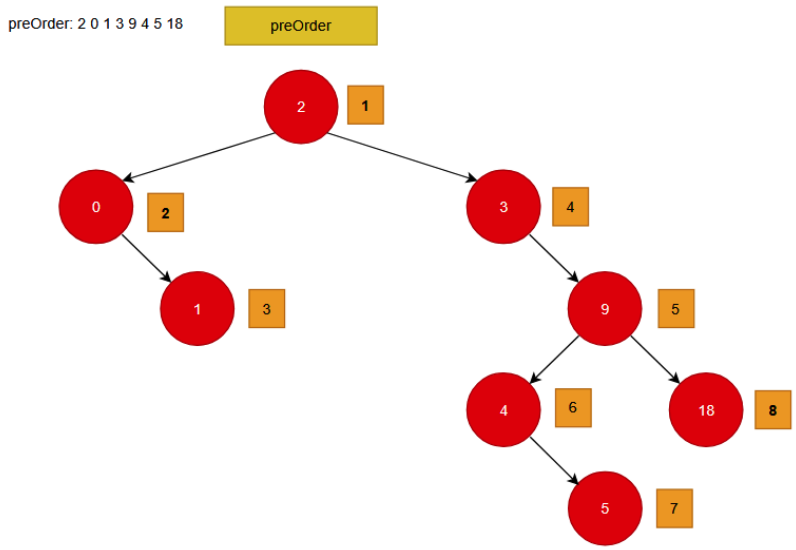
Table 9-6

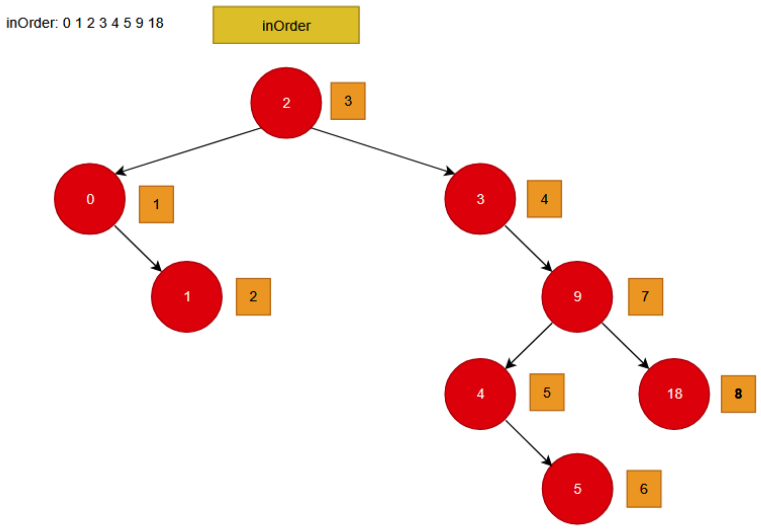
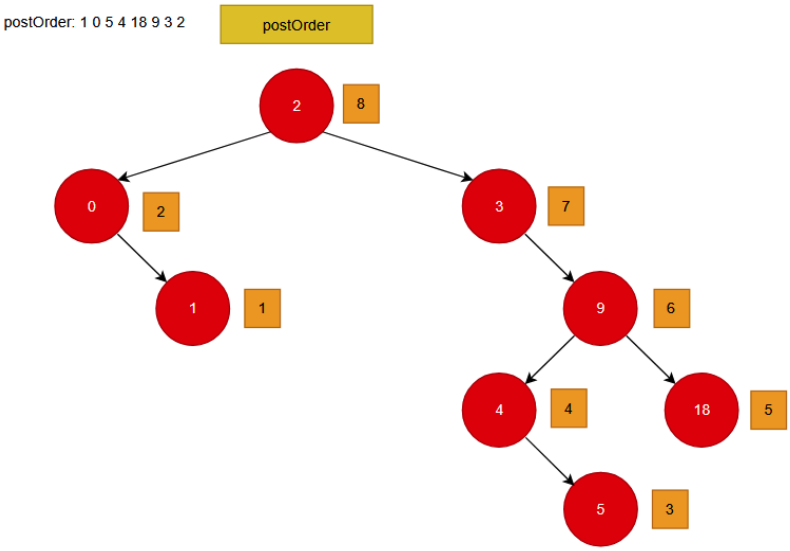
Code Snippet	
Output	
Observations	findData is a small wrapper that resets foundFlag to false and then calls searchHelper with the root, chosen traversal order, and target key. Resetting the flag each time makes sure a previous search doesn't affect the next one.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

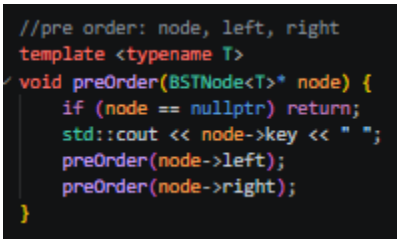
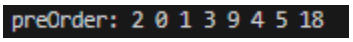
Copy and paste the questions from the lab and include your answers after.

Traversal	Diagram

Tree Diagram	
Pre-order	

In-order	
Post-order	

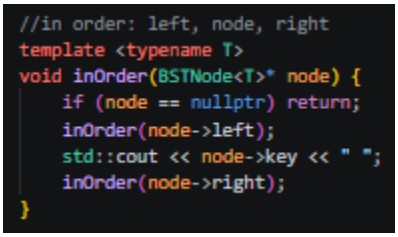
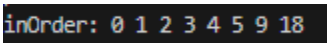
Pre order

Code Snippet	
Output	

```
//pre order: node, left, right
template <typename T>
void preOrder(BSTNode<T>* node) {
    if (node == nullptr) return;
    std::cout << node->key << " ";
    preOrder(node->left);
    preOrder(node->right);
}
```

```
preOrder: 2 0 1 3 9 4 5 18
```

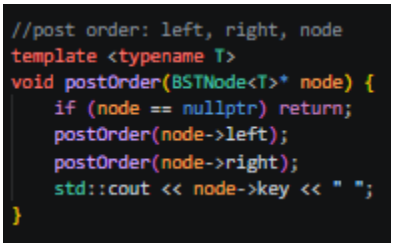
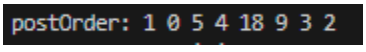
In order

Code Snippet	
Output	

```
//in order: left, node, right
template <typename T>
void inOrder(BSTNode<T>* node) {
    if (node == nullptr) return;
    inOrder(node->left);
    std::cout << node->key << " ";
    inOrder(node->right);
}
```

```
inOrder: 0 1 2 3 4 5 9 18
```

Post order

Code Snippet	
Output	

```
//post order: left, right, node
template <typename T>
void postOrder(BSTNode<T>* node) {
    if (node == nullptr) return;
    postOrder(node->left);
    postOrder(node->right);
    std::cout << node->key << " ";
}
```

```
postOrder: 1 0 5 4 18 9 3 2
```

Observations	For this supplementary activity, I made a simple BST using left and right pointers to show the three traversals. Pre-order visits the root first, in-order visits left then root then right which gives a sorted list, and post-order visits the root last.

Conclusion	In conclusion, this activity helped me understand how tree data structures work in C++, even though it was confusing at first to tell GT, BT, and BST apart. The BST was the easiest for me to understand since it only uses left and right pointers, so the traversals made more sense compared to the general tree. Doing the pre-order, in-order, and post-order traversals for both the GT and BST, plus building the search function, helped me see how the same traversal idea just changes depending on the tree structure. This activity gave me a better grasp of how trees and their traversals actually work in code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.